Faculty of Engineering and Technology

Electrical and Computer Engineering Department

ADVANCED DIGITAL DESIGN

ENCS3310

Project Report

Prepared by:

Al-zahra Nassif 1220168

Instructor: **Abdellatif Abu-Issa**

Section: **3**

Date: 24/12/2024

Abstract

This project focuses on design a comparator for 6 bit signed and unsigned number structurally by using the basic gates (INV, NAND, NOR, AND, OR, XNOR and XOR), then define delay for each gate based on the ID number, and make the circuit synchronous by adding registers to the inputs/outputs, also to write a complete code for verification to discover the errors.

Al_zahra Nassif
1220168

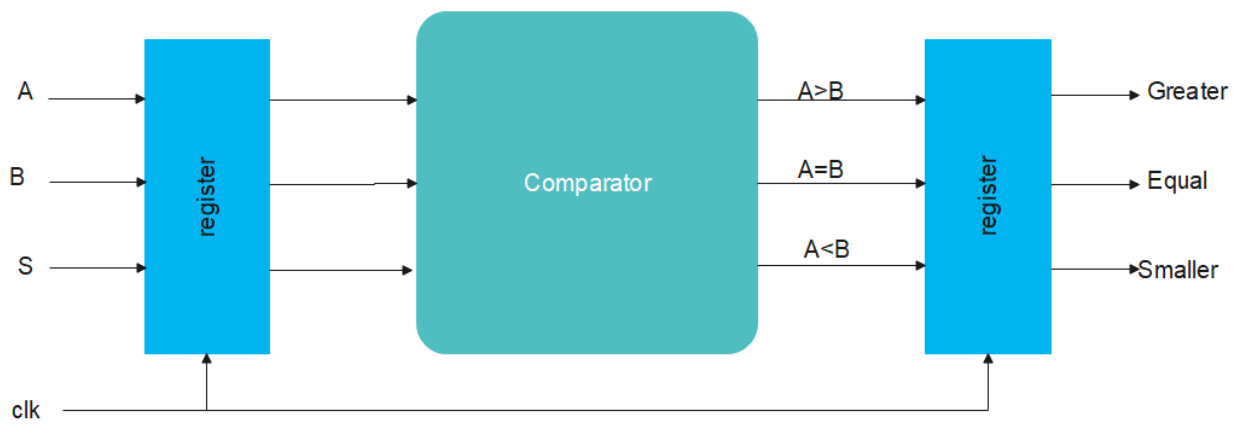

Table of Contents

Abstract I
Table of Contents II
Table of Figures III
List of Tables IV
1. Theoretical overview & Design philosophy 1
1.1 Unsigned Comparator 1
1.2 Signed Comparator 3
1.3 Synchronous Comparator for both signed & unsigned 4
1.4 Behavior Comparator 5
2. Test bench & simulations 6
Conclusion & Future works 10

Table of Figures

Figure 1: 6 bit Comparator diagram.[1] 1
Figure 2: Unsigned comparator module 2
Figure 3: Signed Comparator 3
Figure 4:Synchronous Comparator. 4
Figure 5: Behavior Comparator. 5
Figure 6: Test bench. 6
Figure 7:Results. 8

List of Tables

Table 1: Gates delays. 2

1. Theoretical overview & Design philosophy

At First, To design the comparator for both signed and unsigned numbers, I divided the design into two modules, one for unsigned numbers and the other for signed numbers, then I merge both using third module with “S” to select between them and the registers to synchronize. After that I designed another Comparator but this time behaviorally to use it for verification in the test bench.

1.1 Unsigned Comparator

At First, to design it structurally I used the circuit diagram to know the needed gates, you can see it below at figure 1, note that in the figure Xnor gate is written as (AB’ + A’B)’ which equivalent to (AB+A’B’) == Xnor that I used.

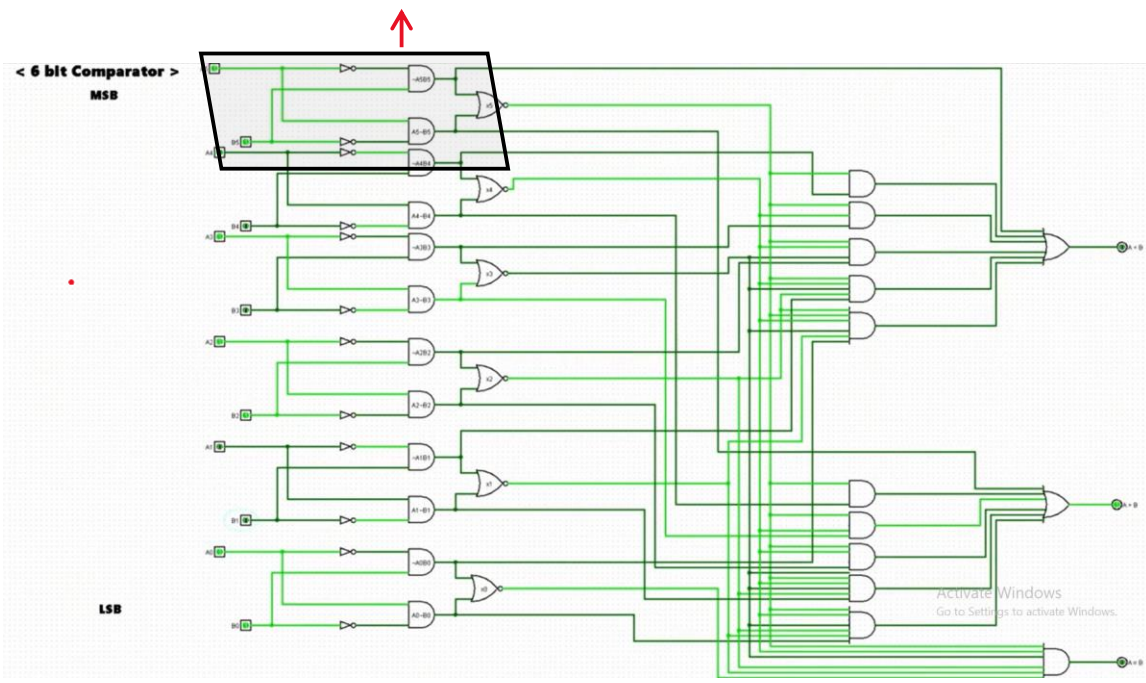

Figure 1: 6 bit Comparator diagram.[1]

[1]: https://www.youtube.com/watch?v=2a5SEGptREM

```
module unsignedComp (A,B,Equal,Greater,Smaller);
    input [5:0] A,B;
    output Equal,Greater,Smaller;
    // Equality comparison A=B
    wire [5:0] X;
    genvar i;
    generate
    for (i=5;i>=0;i=i-1)begin : compare_bits
         xnor #10 (X[i],A[i],B[i]);
         end
    endgenerate

    and #9(Equal,X[5],X[4],X[3],X[2],X[1],X[0]);

    wire [5:0] notB;
    wire [5:0] notA;
    genvar ii;
    generate
    for (ii=5;ii>=0;ii=ii-1)begin : invert_bitsBA
        not #4 (notB[ii],B[ii]);
        not #4 (notA[ii],A[ii]);
         end
    endgenerate

    // Greater comparison A > B
    wire [5:0] Y;

    and #9 (Y[5],A[5],notB[5]);
    and #9 (Y[4],X[5],A[4],notB[4]);
    and #9 (Y[3],X[5],X[4],A[3],notB[3]);
    and #9 (Y[2],X[5],X[4],X[3],A[2],notB[2]);
    and #9 (Y[1],X[5],X[4],X[3],X[2],A[1],notB[1]);
    and #9 (Y[0],X[5],X[4],X[3],X[2],X[1],A[0],notB[0]);
    or #9 (Greater,Y[5], Y[4],Y[3],Y[2],Y[1],Y[0]);

    // Smaller comparison A < B
    wire [5:0] Z;


    and #9 (Z[5],notA[5],B[5]);
    and #9 (Z[4],X[5],notA[4],B[4]);
    and #9(Z[3],X[5],X[4],notA[3],B[3]);
    and #9 (Z[2],X[5],X[4],X[3],notA[2],B[2]);
    and #9 (Z[1],X[5],X[4],X[3],X[2],notA[1],B[1]);
    and #9 (Z[0],X[5],X[4],X[3],X[2],X[1],notA[0],B[0]);

    or #9 (Smaller,Z[5],Z[4],Z[3],Z[2],Z[1],Z[0]);

endmodule
```

Figure 2: Unsigned comparator module

I started with (A=B), using generate to repeat it 6 times for each bit followed by and gate to know if all the bits are equal. Next one is (A>B) means (A notB) which I used the wire X (equal bits) to evaluate it, but the results of each bit greater in Wire 'Y', then used or gate to know if one/many bits are greater, last is (A<B) which its gates same as (A>B) but with "notA, B".

After I build it, I defined the delays for each gate based on number 8 (last bit of my id 1220168) from the given table below.

Gate\ID	0	1	2	3	4	5	6	7	8	9
INV	1	2	1	2	3	4	3	1	4	2
NAND	2	3	3	4	4	5	5	5	6	7
NOR	2	3	4	4	4	5	5	5	6	7
AND	3	4	4	5	6	7	8	6	9	8
OR	3	4	5	5	6	7	8	6	9	8
XNOR	4	5	6	7	8	9	10	8	10	9
XOR	5	6	7	8	9	10	12	9	13	10
Your UDP (Max inputs = 4)	6	7	8	9	10	11	12	10	13	12

Table 1: Gates delays.

1.2 Signed Comparator

```
module signedComp (A, B, Equal, Greater, Smaller);
    input [5:0] A, B;
    output Equal, Greater, Smaller;

    // Equality comparison A=B
    wire [5:0] X;
    genvar i;
    generate
        for (i = 5; i >= 0; i = i - 1) begin : compare_bits
            xnor #10(X[i], A[i], B[i]);
        end
    endgenerate
    and #9 (Equal,X[5],X[4],X[3],X[2],X[1],X[0]);

     wire [5:0] notB;
     wire [5:0] notA;
     genvar ii;
     generate
     for (ii=5;ii>=0;ii=ii-1)begin : invert_bitsBA
          not #4 (notB[ii],B[ii]);
          not #4 (notA[ii],A[ii]);
           end
     endgenerate

    // Greater comparison A > B
    wire [5:0] Y;
    // First compare the sign bits (MSB)
    and #9(Y[5], notA[5], B[5]);   // A positive, B negative
    and #9(Y[4], X[5], A[4], notB[4]);
    and #9(Y[3], X[5], X[4], A[3], notB[3]);
    and #9(Y[2], X[5], X[4], X[3], A[2], notB[2]);
    and #9(Y[1], X[5], X[4], X[3], X[2], A[1], notB[1]);
    and #9(Y[0], X[5], X[4], X[3], X[2], X[1], A[0], notB[0]);
    or #9(Greater,Y[5], Y[4],Y[3],Y[2],Y[1],Y[0]);

    // Smaller comparison A < B
    wire [5:0] Z;

    // First compare the sign bits (MSB)
    and #9(Z[5], A[5], notB[5]);   // A negative, B positive
    and #9(Z[4], X[5], notA[4], B[4]);
    and #9(Z[3], X[5], X[4],  notA[3],B[3]);
    and #9(Z[2], X[5], X[4], X[3], notA[2], B[2]);
    and #9(Z[1], X[5], X[4], X[3], X[2], notA[1], B[1]);
    and #9(Z[0], X[5], X[4], X[3], X[2], X[1], notA[0], B[0]);

    or #9 (Smaller,Z[5],Z[4],Z[3],Z[2],Z[1],Z[0]);

endmodule
```

Figure 3: Signed Comparator

As you can see above:

- A=B is same as unsigned numbers
- A>B, the difference is the MSB, greater means A positive (MSB=0, notA) and B negative(MSB=1,B) So the first operation is “and #9(Y[5], notA[5], B[5]);” and the remaining is same as unsigned.
- A<B, , the difference is the MSB, smaller means A negative (MSB=1 ,A) and B positive(MSB=0,notB). So the first operation is “and #9(Z[5], A[5], notB[5]);” and the remaining is same as unsigned.

1.3 Synchronous Comparator for both signed & unsigned

```
module comparator (A, B, S, clk, Equal, Greater, Smaller);
    input [5:0] A, B;
     reg [5:0] AD,BD;
     reg SD;
    input S, clk;
    output reg Equal, Greater, Smaller;
    always @(posedge clk) begin
         AD <= A;
         BD<= B;
         SD<=S;
     end
    wire ES, GS, SS;   // Outputs from signed comparison
    wire EU, GU, SU;   // Outputs from unsigned comparison

    signedComp sc (.A(AD), .B(BD), .Equal(ES), .Greater(GS), .Smaller(SS));
    unsignedComp uc (.A(AD), .B(BD), .Equal(EU), .Greater(GU), .Smaller(SU));

    always @(posedge clk) begin
        if (SD == 1'b1) begin
            Equal <= ES;
            Greater <= GS;
            Smaller <= SS;
        end else begin
            Equal <= EU;
            Greater <= GU;
            Smaller <= SU;
        end
end
endmodule
```

Figure 4:Synchronous Comparator.

➢ Selection S

I called instance of the signed comparator and another instance of the unsigned comparator both have different output wires, and through the input selection “S”, I assigned the right value to the outputs (Equal, Greater, Smaller).

➢ Synchronization

Through the input clk (clock) the comparator became synchronous, the input can go in only when positive edge clk (0→1) Also the output can go out. Control the time for clk will help to avoid glitches because of the gates delays.

1.4 Behavior Comparator

```
module Comp (AD,BD,SD,clk ,Equal, Greater, Smaller);
input  [5:0] AD,BD;
reg  [5:0] A,B;
reg S;
input SD,clk;
reg EC,GC,SC;
output reg  Equal,Greater,Smaller;
 always @(posedge clk) begin
         A <= AD;
         B<= BD;
         S<=SD;
    end
     always @ (A or B or S) begin
        // Reset output signals
        EC = 0;
        GC = 0;
        SC = 0;
        if (S == 1) begin// Signed comparison
            if ($signed(A) == $signed(B)) EC = 1;
            else if ($signed(A) > $signed(B))GC = 1;
            else if ($signed(A) < $signed(B))SC = 1;

        end
        else begin // Unsigned comparison
            if (A == B) EC = 1;
            else if (A > B)GC = 1;
            else SC = 1;
        end
     end
   always @(posedge clk) begin
             Equal <= EC;
             Greater <= GC;
            Smaller <= SC;
             end
endmodule
```

Figure 5: Behavior Comparator.

I built another comparator behaviorally to use it in the test bench for verification as its outputs will be the expected outputs so I can discover if there is an errors in the structurally Comparator. Also I added registers to the inputs/outputs of the circuit to make the circuit synchronous. So it can match the structurally Comparator outputs to compare them.

2. Test bench & simulations

```
module tb_comparator;

    reg [5:0] A, B;
    reg S, clk;


    wire Equal, Greater, Smaller;
    wire ExpectedEqual, ExpectedGreater, ExpectedSmaller;


    comparator Structural (.A(A), .B(B), .clk(clk), .S(S), .Equal(Equal), .Greater(Greater), .Smaller(Smaller));
    Comp expectedComp (.AD(A), .BD(B), .SD(S), .clk(clk), .Equal(ExpectedEqual), .Greater(ExpectedGreater), .Smaller(ExpectedSmaller));

    integer error_count=0;
    initial begin
        clk = 1'b1;
        forever #14 clk = ~clk;
    end


    initial begin
        // Display header for inputs table
        $display("time     A           B        S  clk  Equal Greater  Smaller");
        $display("-----------------------------------------------------------------------");

         $monitor("%0d    %b      %b      %b     %b   %b      %b      %b", $time, A, B, S, clk, Equal, Greater, Smaller);


        // Initialize inputs
        A = 6'b000000;
        B = 6'b000000;


         // Initialize inputs
         A = 6'b000000;
         B = 6'b000000;
         S = 1'b1;

          repeat(63)begin
               #28 A= A +1;
               repeat(63)begin
                   #28 B= B +1;
                   #28 S =~S;
                   if ((Equal != ExpectedEqual) || (Greater != ExpectedGreater) || (Smaller != ExpectedSmaller)) begin
                   error_count = error_count + 1;
                   end
               end
               end

           if (error_count > 0)
             $display("\nFailed !-_-!,Total errors:%d\n",error_count);
          else
             $display("\nAll tests passed !><!\n");

         $finish;
    end

endmodule
```

Figure 6: Test bench.

Let's discuss some points:

➢ **Test cases coverage**

As you can see above, I covered all the possible cases using repeat (63) twice, each value of A will compare with all possible values of B two times, one when S=0 unsigned and the other when S=1 signed.

```
repeat(63)begin
    #28 A= A +1;
    repeat(63)begin
        #28 B= B +1;
        #28 S =~S;
```

➢ **verification**

I used the comparator which I design it behaviorally to be the expected truth results as I mentioned in its part, However, I also defined a counter for the errors, and after each case I checked the validity of results out from the structurally comparator with the expected result, if they are not the same, the error count will increase and after the end of all cases based on this error counter, a statement will be written on console screen .

```
        if ((Equal != ExpectedEqual) || (Greater != ExpectedGreater) || (Smaller != ExpectedSmaller)) begin
        error_count = error_count + 1;
        end
    end
    end

 if (error_count > 0)
   $display("\nFailed !-_-!,Total errors:%d\n",error_count);
else
   $display("\nAll tests passed !><!\n");
```

➢**clk and inputs delay**

since all gates I used has a delay, I had to find minimum delay that will not cause a glitch, so I can observe an accurate results, I made this process by tuning. I write a 10 cases to try on and I started the clk delay by 4 and by each run i checked the results if it contain an error(glitch), if it contain I increased the delay and so on until I reached a result without glitch. This happened at 14. Note that 14 is the time to toggle the clk this mean that a posedge will came after 28 (1→0→1 || 0→1→0). And we need the inputs on each posedge: so I put the delay for inputs 28 as you can see above, and the output out after 28.

```
integer error_count=0;
initial begin
    clk = 1'b1;
    forever #14 clk = ~clk;
end
```

➢**simulation Results**

1- I will simulate it and you will see what the console screen will appear
2- I will discuss some result
3- I will introduce an error and we will see what will be written on the console screen

- Don’t forget what each column indicates to

  “time A B S clk Equal Greater Smaller"

1- as you can see “All tests passed !><!”

```
KERNEL: 223622    111111    111010    0    0    0    1    0
KERNEL: 223636    111111    111010    1    1    0    1    0
KERNEL: 223650    111111    111010    1    0    0    1    0
KERNEL: 223664    111111    111011    1    1    0    1    0
KERNEL: 223678    111111    111011    1    0    0    1    0
KERNEL: 223692    111111    111011    0    1    0    1    0
KERNEL: 223706    111111    111011    0    0    0    1    0
KERNEL: 223720    111111    111100    0    1    0    1    0
KERNEL: 223734    111111    111100    0    0    0    1    0
KERNEL: 223748    111111    111100    1    1    0    1    0
KERNEL: 223762    111111    111100    1    0    0    1    0
KERNEL: 223776    111111    111101    1    1    0    1    0
KERNEL: 223790    111111    111101    1    0    0    1    0
KERNEL: 223804    111111    111101    0    1    0    1    0
KERNEL: 223818    111111    111101    0    0    0    1    0
KERNEL: 223832    111111    111110    0    1    0    1    0
KERNEL: 223846    111111    111110    0    0    0    1    0
KERNEL: 223860    111111    111110    1    1    0    1    0
KERNEL: 223874    111111    111110    1    0    0    1    0
KERNEL: 223888    111111    111111    1    1    0    1    0
KERNEL: 223902    111111    111111    1    0    0    1    0
KERNEL: 223916    111111    111111    0    1    1    0    0
KERNEL: 223930    111111    111111    0    0    1    0    0
KERNEL: 223944    111111    000000    0    1    1    0    0
KERNEL: 223958    111111    000000    0    0    1    0    0
KERNEL: 223972    111111    000000    1    1    0    1    0
KERNEL: 223986    111111    000000    1    0    0    1    0
KERNEL: 224000    111111    000001    1    1    0    0    1
KERNEL: 224014    111111    000001    1    0    0    0    1
KERNEL:
KERNEL: All tests passed !><!
```

Figure 7:Results.

2- Lets take this case

```
221984    111111    011101    1    1    0    0    1
221998    111111    011101    1    0    0    0    1
222012    111111    011101    0    1    0    0    1
222026    111111    011101    0    0    0    0    1
222040    111111    011110    0    1    0    1    0
```

221984 : A=111111 B=011101 S=1(signed) clk= 1 there is a posedge this mean that the inputs will enter. Note that 0 0 1 is not the result for this case it’s the result for the previous case

221998: clk=0(1→0)negedge means that the output and the input not change .

222012: which is after 28 from 221984

At first look at the results 0 01 means that A is smaller than B which is true

Then look to the input A=111111 B=011101 S=1(unsigned) and here A>B

After 28 on the next posedge time **222040** : the output is 010 means A greater which is true.

Another case

```
220220    111110    111110    0    1    0    1    0
220234    111110    111110    0    0    0    1    0
220248    111110    111110    1    1    1    0    0
220262    111110    111110    1    0    1    0    0
220276    111110    111111    1    1    1    0    0
```

220220 : A=111110 B= 111110 S=0 unsigned here A=B

220248: the output is 100 means A=B which is true and the new input is A=111110/B=111110 S=1 signed

220276: the output is 100 means A=B which is true .

Another case

195020	110111	111111	0	1	0	0	1
195034	110111	111111	0	0	0	0	1
195048	110111	000000	0	1	0	0	1
195062	110111	000000	0	0	0	0	1
195076	110111	000000	1	1	0	1	0
195090	110111	000000	1	0	0	1	0
195104	110111	000001	1	1	0	0	1

195020: A= 110111/B=111111 S=0 unsigned A<B

195048: the output is 001 means A<B which is true

195076: A= 110111/B=000000 S=1 signed A<B , again the output for this time is for the previous input

195104: the output is 001 means A<B

Here is another example for signed and greater

198128	111000	110110	1	1	0	1	0
198142	111000	110110	1	0	0	1	0
198156	111000	110111	1	1	0	1	0

3- I Introduced an error in the signed Comparator, used and gate instead of or

```
// Greater comparison A > B
wire [5:0] Y;
// First compare the sign bits (MSB)
and #9(Y[5], notA[5], B[5]);   // A positive, B negative
and #9(Y[4], X[5], A[4], notB[4]);
and #9(Y[3], X[5], X[4], A[3], notB[3]);
and #9(Y[2], X[5], X[4], X[3], A[2], notB[2]);
and #9(Y[1], X[5], X[4], X[3], X[2], A[1], notB[1]);
and #9(Y[0], X[5], X[4], X[3], X[2], X[1], A[0], notB[0]);
//or #9(Greater,Y[5], Y[4],Y[3],Y[2],Y[1],Y[0]);
 and  #9(Greater,Y[5], Y[4],Y[3],Y[2],Y[1],Y[0]);//intoduce an error
```

The console screen showed that there is errors with its number

223818	111111	111101	0	0	0	0	0
223832	111111	111110	0	1	0	1	0
223846	111111	111110	0	0	0	1	0
223860	111111	111110	1	1	0	1	0
223874	111111	111110	1	0	0	1	0
223888	111111	111111	1	1	0	0	0
223902	111111	111111	1	0	0	0	0
223916	111111	111111	0	1	1	0	0
223930	111111	111111	0	0	1	0	0
223944	111111	000000	0	1	1	0	0
223958	111111	000000	0	0	1	0	0
223972	111111	000000	1	1	0	1	0
223986	111111	000000	1	0	0	1	0
224000	111111	000001	1	1	0	0	1
224014	111111	000001	1	0	0	0	1

Failed !-_-!,Total errors: 993

some faulty results!

198128	111000	110110	1	1	0	1	0
198142	111000	110110	1	0	0	1	0
198156	111000	110111	1	1	0	0	0
198170	111000	110111	1	0	0	0	0
198184	111000	110111	0	1	0	0	0

Conclusion & Future works

After this project and after writing the codes, and understanding the work behind them, I now know how to design a 6 bit Comparator structurally and behaviorally, I understand how to deal with delays and glitches and how to make the circuit synchronous using clk. I also learned how to design a block diagram. The results accurate highlight the importance of choosing the right clk delay. In short, we became more familiar with Active_HDL and writing codes, make a delay and testing systems. I think my future works to improve this system by reduce the number of gates used by taking advantage of each gate I write before, for example in signed Comparator module, I could use the gates for unsigned without writing them again.